

CS3212 Algorithms

Aya Zirikly
09/22/2026

Quiz 2

09/24/2026

What is included: 09/08 till 09/17 (09/17 is included)

Summary

Operation	Best Case (Balanced)	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Space Complexity	$O(n)$ total space	$O(n)$ total space	$O(n)$ total space ($O(n)$ call stack)

Summary

Operation	Best Case (Balanced)	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(l)$	Memory consumed by recursive calls Every time a recursive function calls itself, the computer pushes a "stack frame" into execution memory to track the function's variables and return address.
Deletion	$O(\log n)$	$O(l)$	
Space Complexity	$O(n)$ total space	$O(n)$ total space	$O(n)$ total space ($O(n)$ call stack)

To prevent binary search trees from degenerating into linear paths, self-balancing trees (such as AVL trees or Red-Black trees)

The AVL Goal

- ❑ AVL trees are standard BSTs with a strict **height invariant**
 - ❑ keep the tree roughly dynamic-balanced after every insertion so height stays **$O(\log n)$** .

Introduction & Origin of AVL Trees

1962

First Self-Balancing BST

Invented by Soviet mathematicians

Georgy Adelson-Velsky and
Evgenii Landis.

$O(\log N)$

Strict Height Bound

Guarantees logarithmic search time
by dynamic structural rebalancing.

Paper

Milestone Publication

Published in *"An algorithm for the
organization of information"*.

Motivation: The Degenerate BST Problem

Standard BST (Sequential Insert)

Inserting 10, 20, 30, 40 creates a degenerate linked list.

```
10 → 20 → 30 → 40  
Height = N - 1  
Search Time:  $O(N)$ 
```

AVL Tree (Balanced Invariant)

Rotations restructure nodes during insertion.

```
  20  
 /  \  
10   30 → 40  
Height  $\leq 1.44 \log N$   
Search Time:  $O(\log N)$ 
```

Searching degrades from logarithmic time to linear time.

Measuring Balance — The Balance Factor

Quantitative Balance Metric:

- To maintain $O(\log n)$ tree height, the AVL algorithm uses the **Balance Factor** to measure height differences across subtrees.
- For any node x :

$$\text{Balance Factor}(x) = \text{height}(\text{left_child}) - \text{height}(\text{right_child})$$

The AVL Invariant Rule:

- For **every** node x in a valid AVL tree, the Balance Factor must belong to the set $\{-1, 0, +1\}$:
 - **-1**: Right-heavy (right subtree is 1 level taller)
 - **0**: Perfectly balanced (both subtrees have equal height)
 - **+1**: Left-heavy (left subtree is 1 level taller)

Triggering Rebalance Operations:

- When an insertion or deletion causes any node's Balance Factor to reach **+2** or **-2**, an imbalance is detected.
- The algorithm immediately performs local **tree rotations** at that node to restore the Balance Factor to the valid range.

Balance Factor: $BF(v) = \text{height}(\text{left}) - \text{height}(\text{right})$

Balanced Range

$BF \in \{-1, 0, +1\}$

Left Heavy Imbalance

$BF = +2$

Right Heavy Imbalance

$BF = -2$

An AVL tree has three primary conditions that must hold true at all times:

1. The Binary Search Tree (BST) Property (Ordering Condition)

- ★ An AVL tree is first and foremost a Binary Search Tree. For every node x :
 - All keys in the left subtree must be strictly smaller than x 's key ($\text{key} < x.\text{key}$).
 - All keys in the right subtree must be strictly larger than x 's key ($\text{key} > x.\text{key}$).
 - An in-order traversal of the tree always yields keys in sorted, ascending order.

An AVL tree has three primary conditions that must hold true at all times:

2. The Structural Binary Tree Condition

- ★ Every node can have **at most two children** (a left child and a right child).
- ★ There are no cycles, and every node (except the root) has exactly one parent.

An AVL tree has three primary conditions that must hold true at all times:

3. Strict Height Definition Condition

- ★ The height of a leaf node is typically defined as 1 (or 0, depending on convention).
- ★ The height of an empty tree/subtree (NULL / None) is defined as 0 (or -1).
- ★ For every non-empty node x, its height must strictly satisfy:

$$\text{height}(x) = 1 + \max(\text{height}(x.\text{left}), \text{height}(x.\text{right}))$$

Property	Standard BST	AVL Tree
Ordering	$\text{key}(\text{left}) < \text{key}(\text{node}) < \text{key}(\text{right})$	$\text{key}(\text{left}) < \text{key}(\text{node}) < \text{key}(\text{right})$
Max Children per Node	2	2
Height Difference (BF)	Unbounded (can be $n-1$)	Strictly $\in \{-1, 0, +1\}$
Worst-Case Search Time	$O(n)$ (degenerated line)	Guaranteed $O(\log n)$

AVL Tree Node Structure

BST Ordering Invariant: Left subkeys < Node key < Right subkeys.

Explicit Height Field: Each node stores *height* to allow $O(1)$ balance calculation.

Recursive Subtrees: Every subtree inside an AVL tree is also an AVL tree.

Why storing height?

- ❑ **Without storing height:** To figure out a subtree's height, you would have to recursively visit **every single node** inside that subtree. Calculating height for a subtree with k nodes would take $O(k)$ time. Doing this at every step up the tree would slow down insertions from $O(\log n)$ to $O(n)$.
- ❑ **With stored height:** Computing the Balance Factor requires looking at just two numbers (`left.height` and `right.height`). This lookup takes **$O(1)$ constant time**.

AVL Tree Node Structure

C

```
struct AVLNode {  
    int key;  
    int height;  
    AVLNode* left;  
    AVLNode* right;  
};
```

Python

```
class AVLNode:  
  
    def __init__(self, key):  
        self.key = key # Stored value / payload  
        self.height = 1 # Subtree height (new leaf nodes start at height 1)  
        self.left = None # Reference to left child  
        self.right = None # Reference to right child
```

Finding the "Worst-Case" AVL Tree

1. Finding the "Worst-Case" AVL Tree

To find the maximum height h for an AVL tree with n nodes, we calculate the minimum number of nodes $N(h)$ required to build an AVL tree of height h .

For a tree of height h to have as few nodes as possible while maintaining the balance property (height difference between subtrees ≤ 1):

- One subtree must have height $h - 1$.
- The other subtree must have height $h - 2$.

Adding the root node gives the recursive formula: $N(h) = 1 + N(h - 1) + N(h - 2)$

Finding the "Worst-Case" AVL Tree

2. The Fibonacci Connection

- ❑ This relationship closely mirrors the Fibonacci sequence

Finding the "Worst-Case" AVL Tree

2. The Fibonacci Connection

- ❑ This relationship closely mirrors the Fibonacci sequence $F(n) = F(n-1) + F(n-2)$

The Fibonacci Connection

Let $N(h)$ be the minimum number of nodes needed to make an AVL tree of height h .

- **Height 0** (empty tree): Requires **0** nodes.
 - $N(0) = 0$
- **Height 1** (single root node): Requires **1** node.
 - $N(1) = 1$
- **Height 2** (root + 1 child): Requires **2** nodes.
 - $N(2) = 1 + N(1) + N(0) = 1 + 1 + 0 = 2$
- **Height 3** (root + height 2 subtree + height 1 subtree): Requires **4** nodes.
 - $N(3) = 1 + N(2) + N(1) = 1 + 2 + 1 = 4$
- **Height 4**: Requires **7** nodes.
 - $N(4) = 1 + N(3) + N(2) = 1 + 4 + 2 = 7$

The Fibonacci Connection

Comparing $N(h)$ to Fibonacci Numbers

Compare the sequence of $N(h)$ directly against $\text{Fibonacci}(h + 2)$:

Height (h)	$N(h)$ (Min Nodes)	$\text{Fibonacci}(h + 2)$	Relationship
0	0	$\text{Fibonacci}(2) = 1$	$1 - 1 = 0$
1	1	$\text{Fibonacci}(3) = 2$	$2 - 1 = 1$
2	2	$\text{Fibonacci}(4) = 3$	$3 - 1 = 2$
3	4	$\text{Fibonacci}(5) = 5$	$5 - 1 = 4$
4	7	$\text{Fibonacci}(6) = 8$	$8 - 1 = 7$
5	12	$\text{Fibonacci}(7) = 13$	$13 - 1 = 12$

The Fibonacci Connection

Comparing $N(h)$ to Fibonacci Numbers

What do you notice?

Compare the sequence of $N(h)$ directly against $\text{Fibonacci}(h + 2)$:

Height (h)	$N(h)$ (Min Nodes)	$\text{Fibonacci}(h + 2)$	Relationship
0	0	$\text{Fibonacci}(2) = 1$	$1 - 1 = 0$
1	1	$\text{Fibonacci}(3) = 2$	$2 - 1 = 1$
2	2	$\text{Fibonacci}(4) = 3$	$3 - 1 = 2$
3	4	$\text{Fibonacci}(5) = 5$	$5 - 1 = 4$
4	7	$\text{Fibonacci}(6) = 8$	$8 - 1 = 7$
5	12	$\text{Fibonacci}(7) = 13$	$13 - 1 = 12$

The Fibonacci Connection

Comparing $N(h)$ to Fibonacci Numbers

Compare the sequence of $N(h)$ directly against $\text{Fibonacci}(h + 2)$:

Every value of $N(h)$ is exactly **1 less** than $\text{Fibonacci}(h + 2)$.

Height (h)	$N(h)$ (Min Nodes)	$\text{Fibonacci}(h + 2)$	Relationship
0	0	$\text{Fibonacci}(2) = 1$	$1 - 1 = 0$
1	1	$\text{Fibonacci}(3) = 2$	$2 - 1 = 1$
2	2	$\text{Fibonacci}(4) = 3$	$3 - 1 = 2$
3	4	$\text{Fibonacci}(5) = 5$	$5 - 1 = 4$
4	7	$\text{Fibonacci}(6) = 8$	$8 - 1 = 7$
5	12	$\text{Fibonacci}(7) = 13$	$13 - 1 = 12$

Finding the "Worst-Case" AVL Tree

2. The Fibonacci Connection

- ❑ This relationship closely mirrors the Fibonacci sequence $F(n) = F(n-1) + F(n-2)$
- ❑ Solving this recurrence relation: $n(h) = \text{Fibonacci}(h + 2) - 1$
- ❑ Because Fibonacci numbers grow exponentially based on the Golden Ratio ($\phi \approx 1.618$)

The minimum number of nodes grows as: $n \approx (\phi^{(h + 2)} / \sqrt{5}) - 1$

Finding the "Worst-Case" AVL Tree

2. The Fibonacci Connection

- ❑ This relationship closely mirrors the Fibonacci sequence $F(n) = F(n-1) + F(n-2)$
- ❑ Solving this recurrence relation: $n(h) = \text{Fibonacci}(h + 2) - 1$
- ❑ Because Fibonacci numbers grow exponentially based on the Golden Ratio ($\phi \approx 1.618$)

The minimum number of nodes grows as: $n \approx (\phi^{(h + 2)} / \sqrt{5}) - 1$

Binet's Formula for
Fibonacci Numbers

Finding the "Worst-Case" AVL Tree

3. Solving for Height (h)

$$n \approx (\phi^{(h+2)} / \sqrt{5}) - 1$$

$$n + 1 \approx \phi^{(h+2)} / \sqrt{5}$$

$$(n + 1) * \sqrt{5} \approx \phi^{(h+2)}$$

$$\log_2((n + 1) * \sqrt{5}) \approx \log_2(\phi^{(h+2)})$$

$$\log_2(n + 1) + \log_2(\sqrt{5}) \approx (h + 2) * \log_2(\phi)$$

$$(\log_2(n + 1) + \log_2(\sqrt{5})) / \log_2(\phi) \approx h + 2$$

Finding the "Worst-Case" AVL Tree

3. Solving for Height (h)

$$n \approx (\phi^{(h+2)} / \sqrt{5}) - 1$$

$$n + 1 \approx \phi^{(h+2)} / \sqrt{5}$$

$$(n + 1) * \sqrt{5} \approx \phi^{(h+2)}$$

$$\log_2((n + 1) * \sqrt{5}) \approx \log_2(\phi^{(h+2)})$$

$$\log_2(n + 1) + \log_2(\sqrt{5}) \approx (h + 2) * \log_2(\phi)$$

$$(\log_2(n + 1) + \log_2(\sqrt{5})) / \log_2(\phi) \approx h + 2$$

$$1.44042 * \log_2(n + 1) + \text{constant} \approx h + 2$$

$$h < 1.44 * \log_2(n)$$

$$\phi \approx 1.61803$$

$$\log_2(1.61803) \approx 0.69424$$

$$1 / \log_2(\phi) = 1 / 0.69424 \approx 1.44042$$

Because $1 / \log_2(\text{Golden Ratio}) \approx 1.44$, an AVL tree is guaranteed to be at most **44% taller** than a perfectly balanced binary tree in the absolute worst case.

This guarantees strict $O(\log n)$ search times.

AVL Tree Operations

Search (AVL-SEARCH)

Goal: Locate a target key k in the tree.

Mechanism: Identical to standard BST search; traverses down a single path based on key comparisons.

Time Complexity: Guaranteed $O(\log n)$ worst-case runtime because

$$\text{tree height } h \leq 1.44 \log_2 n$$

Search (AVL-SEARCH)

```
FUNCTION search(root, targetKey):  
    currentNode = root  
  
    WHILE currentNode IS NOT NULL:  
        IF targetKey == currentNode.key:  
            RETURN currentNode          // Key found!  
        ELSE IF targetKey < currentNode.key:  
            currentNode = currentNode.left // Move left if target is smaller  
        ELSE:  
            currentNode = currentNode.right // Move right if target is larger  
  
    RETURN NULL                        // Key not found in tree
```

Search (AVL-SEARCH)

```
FUNCTION search(root, targetKey):  
    currentNode = root  
  
    WHILE currentNode IS NOT NULL:  
        IF targetKey == currentNode.key:  
            RETURN currentNode          // Key found!  
        ELSE IF targetKey < currentNode.key:  
            currentNode = currentNode.left // Move left if target is smaller  
        ELSE:  
            currentNode = currentNode.right // Move right if target is larger  
  
    RETURN NULL                        // Key not found in tree
```

Iterative

Search (AVL-SEARCH)

Recursive?

Search (AVL-SEARCH)

```
FUNCTION search(node, targetKey):  
  IF node IS NULL OR targetKey == node.key:  
    RETURN node          // Returns node if found, or NULL if end reached  
  
  IF targetKey < node.key:  
    RETURN search(node.left, targetKey)  
  ELSE:  
    RETURN search(node.right, targetKey)
```

Recursive?

Search (AVL-SEARCH)

Time Complexity: Worst-case $O(\log n)$ because the maximum height is capped at $1.44 * \log_2(n)$.

Space Complexity:

- **Iterative:** $O(1)$ constant space.
- **Recursive:** $O(\log n)$ stack space due to recursion depth.

Insertion

Goal: Add a new key while maintaining both BST order and AVL balance (height difference between subtrees ≤ 1).

3-Step Process:

1. **Insert:** Standard BST insert (walk down to a leaf).
2. **Update:** Walk back up, updating node heights.
3. **Rebalance:** Check balance factors and apply rotations if unbalanced.

Insertion

When a node becomes unbalanced (Balance Factor > 1 or < -1), apply **1 of 4 rotation strategies**:

- **Case 1: Left-Left (LL)**
 - *Trigger*: Inserted into left child's left subtree.
 - *Fix*: **1 Right Rotation** at unbalanced node.
- **Case 2: Right-Right (RR)**
 - *Trigger*: Inserted into right child's right subtree.
 - *Fix*: **1 Left Rotation** at unbalanced node.
- **Case 3: Left-Right (LR)**
 - *Trigger*: Inserted into left child's right subtree.
 - *Fix*: **Double Rotation** (Left Rotate left child $\rightarrow$ Right Rotate unbalanced node).
- **Case 4: Right-Left (RL)**
 - *Trigger*: Inserted into right child's left subtree.
 - *Fix*: **Double Rotation** (Right Rotate right child $\rightarrow$ Left Rotate unbalanced node).

Insertion — Complexity Analysis

Time Complexity:

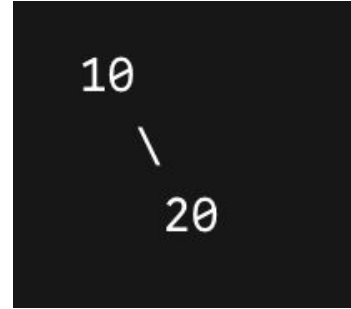
- **Worst Case: $O(\log n)$** — Traversal down to the leaf takes at most $1.44 * \log_2(n)$ steps. Updating heights and performing at most 2 rotations on the way back up takes **$O(1)$** time per level.

Space Complexity:

- **Recursive: $O(\log n)$** — Call stack depth corresponds to tree height.
- **Iterative: $O(\log n)$** — Explicit stack depth corresponds to tree height.

Step 1: Insert 10, 20, 30 (Triggers Right-Right Case)

Insert 10, then 20



Insert 30

```
10 (Balance = -2) [Unbalanced!]
```

```
\
```

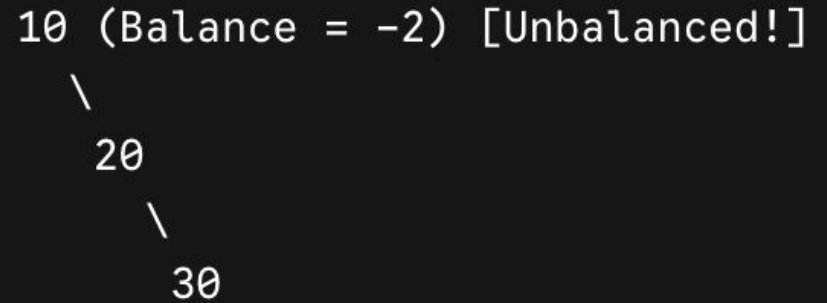
```
20
```

```
\
```

```
30
```

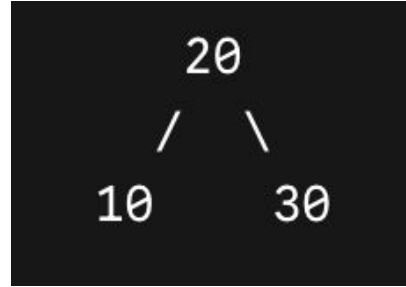
Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).



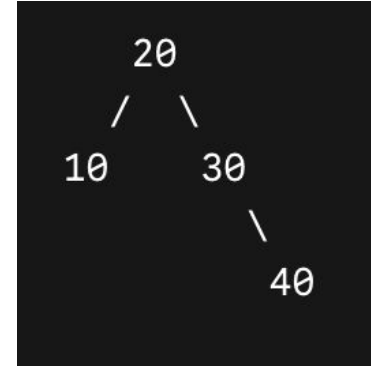
Fix: Left Rotation at 10

Rotate 20 up to become the new root.

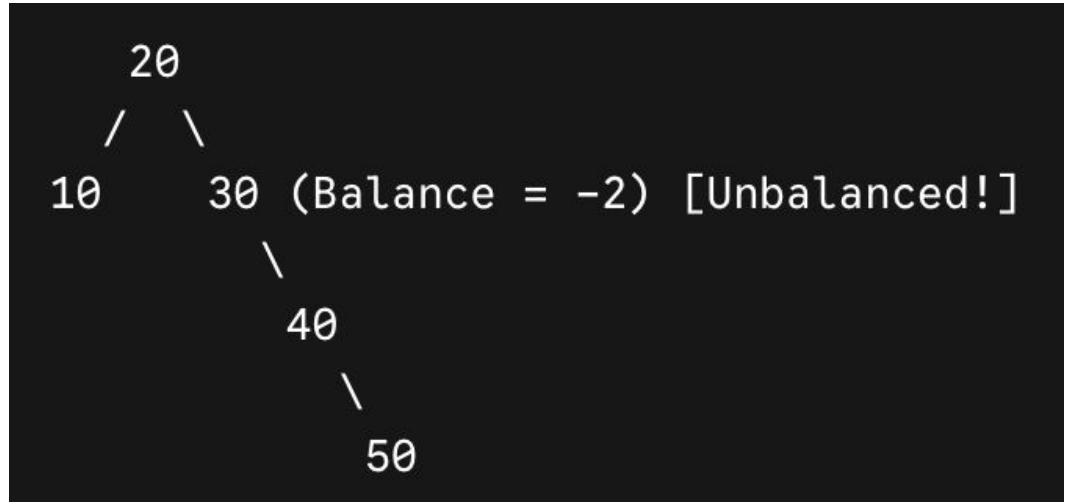


Step 2: Insert 40, 50 (Triggers Right-Right Case)

Insert 40



Insert 50



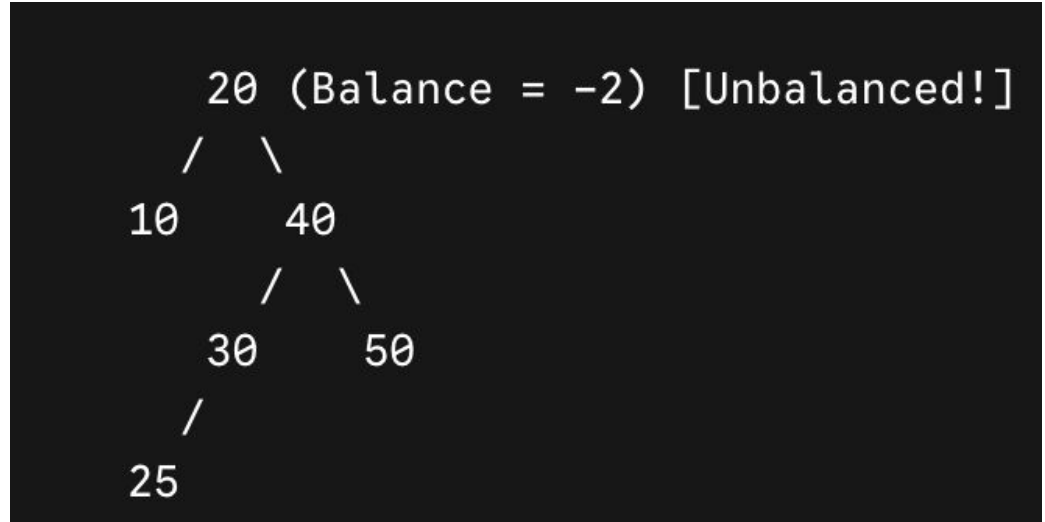
Node 30 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Insert 25 (Triggers Right-Left Case)

Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

Node 20 balance factor becomes -2 (Left Height 1 – Right Height 3).

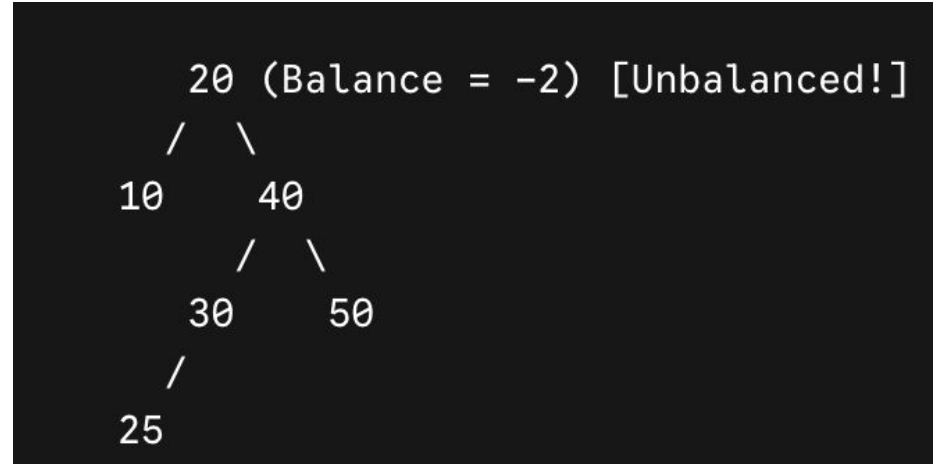


Insert 25 (Triggers Right-Left Case)

Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

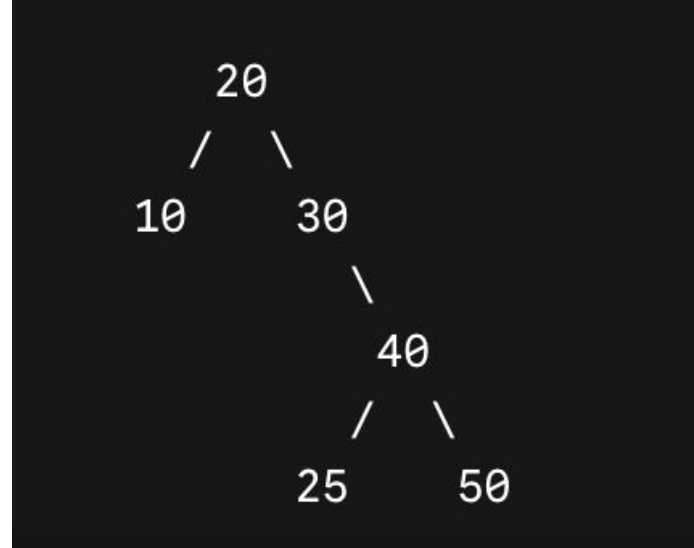
Node 20 balance factor becomes **-2** (Left Height 1 – Right Height 3).



Because 25 was inserted into the **left** subtree of 40's **right** child, this is a **Right-Left (RL) Case**.

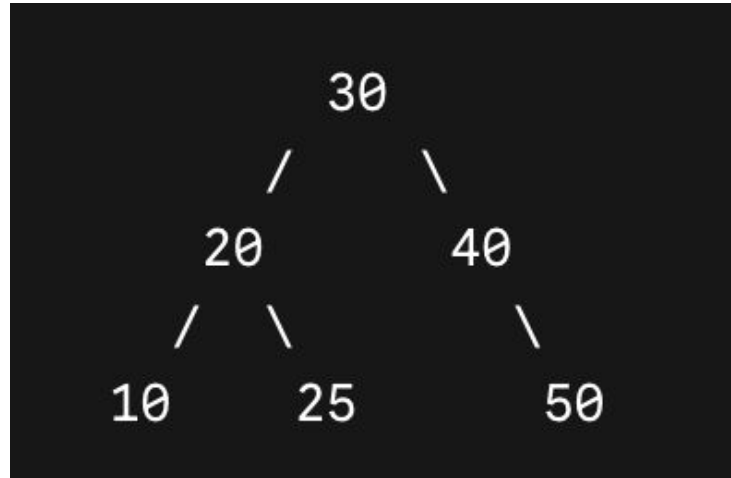
Fix Part 1: Right Rotation at 40 (its right child)

Rotate 30 up



Fix Part 2: Left Rotation at 20

Rotate 30 up to become the new root of the entire tree.



Inserted Key	Unbalanced Node	Balance Factor	Case Identified	Fix Action Taken
30	Node 10	-2	Right-Right (RR)	Single Left Rotation at 10
50	Node 30	-2	Right-Right (RR)	Single Left Rotation at 30
25	Node 20	-2	Right-Left (RL)	Double Rotation: Right Rotate 40, then Left Rotate 20

Deletion

Goal: Remove a key while preserving BST order and height balance.

1. **Delete (Standard BST):**
 - **0 Children (Leaf):** Remove node.
 - **1 Child:** Replace node with its child.
 - **2 Children:** Replace node with its **in-order successor** (smallest node in right subtree), then delete the successor.
2. **Update Heights:**
 - Walk up the tree from the deleted node's parent toward the root, recalculating heights.
3. **Rebalance:**
 - Check balance factors along the path up to the root.
 - If unbalanced, apply the **exact same 4 rotations** (LL, RR, LR, RL) used in insertion.

Deletion vs. Insertion

- ❑ **Insertion:** Requires at most **1 rotation** (single or double) to rebalance the entire tree because tree height increases by at most 1.
- ❑ **Deletion:** May require **up to $O(\log n)$ rotations** cascading all the way up to the root because removing a node can shrink subtree heights at multiple levels.

Deletion Complexity

Time Complexity: Worst-Case $O(\log n)$

Space Complexity: $O(\log n)$ call stack

Why Binary Trees Fail on Disk

- ❑ **The Memory Gap:** Main Memory (RAM) is fast, but Disk Storage (SSD/HDD) operates in large chunks called **blocks or pages** (typically 4KB to 16KB).
- ❑ **The Cost of Disk I/O:** Reading a block from disk takes orders of magnitude longer than accessing RAM.
- ❑ **Why AVL Trees Fall Short for Large Data:**
 - ❑ Binary trees have 2 children per node, creating tall trees with large height: $\text{height} = \log_2(n)$.
 - ❑ Each node access is a separate disk read. A tree with height 30 means **30 distinct disk reads** per search.
- ❑ Instead of making trees *taller*, make them *fatter*!
 - ❑ Fit as many keys into a single node as can fit inside a single 4KB disk page.

Multiway Search Trees (The B-Tree Concept)

Definition: A tree where a single node can store **multiple keys** and have **more than two children** (an m-way search tree).

Key Properties of a B-Tree:

- **Order m:** Every node can have at most m children and m - 1 keys.
- **Self-Balancing:** All leaf nodes are guaranteed to be at the exact same depth.
- **High Branching Factor:** Instead of 2 children, a node might have 100 to 1,000 children.

Impact on Performance:

- Tree height drops dramatically: $\text{height} = \log_m(n)$.
- A database with 1 billion items needs a height of only **3 or 4** in a B-Tree, meaning only 3 to 4 disk reads per query!

B-Tree vs. B+ Tree (The Modern Database Standard)

❑ What is a B-Tree?

- ❑ Both internal nodes and leaf nodes store actual **data records or pointers**.
- ❑ Search can terminate early if the key is found at an internal node.

❑ What is a B+ Tree?

- ❑ **Internal Nodes:** Store *only* routing keys and child pointers (no actual data).
- ❑ **Leaf Nodes:** Store *all* actual data or pointers and are **linked together in a doubly linked list**.

❑ Why Databases (e.g., MySQL, PostgreSQL) Prefer B+ Trees:

- ❑ Higher fan-out (more keys fit into an internal node because there is no data payload).
- ❑ Sequential range scans (e.g., `WHERE age BETWEEN 20 AND 30`) only need to find the first leaf and follow the linked list, avoiding tree re-traversals.

Core Operations Overview (B-Tree / B+ Tree)

Search: Traverse down child pointers based on range comparisons inside each node.

Insertion & Node Splitting:

- Insert key into appropriate leaf.
- If node overflows (more than $m - 1$ keys), **split node into two** and push the middle key up to the parent.
- Splits propagate upwards (the tree grows *up*, not down).

Deletion & Node Merging:

- Remove key from leaf.
- If node underflows (fewer than $\text{ceiling}(m / 2) - 1$ keys), borrow keys from siblings or **merge** with a sibling.

Summary

Feature	Binary Search Tree / AVL	B-Tree	B+ Tree
Max Children / Node	2	m (High)	m (Very High)
Primary Storage Medium	Main Memory (RAM)	Disk / Filesystems	Database Engines / Indexes
Data Storage Location	All Nodes	All Nodes	Leaves Only
Range Queries	Inefficient ($O(n)$ in-order traversal)	Moderate	Optimal (Traverse linked leaf nodes)
Search Time Complexity	$O(\log_2(n))$	$O(\log_m(n))$	$O(\log_m(n))$